香山 Kunminghu V2 自定义 vdot.vv 指令设计与验证

摘要：本项目面向 CIE 全国 RISC-V 高水平创新及应用大赛香山社区 RISC-V 应用创新赛道，基于 XiangShan Kunminghu V2 处理器实现自定义向量点积指令 vdot.vv。指令面向 VLEN=128、SEW=8 的向量状态，仅对两个源向量的低 8 个 signed INT8 元素执行乘加归约，将 signed INT32 结果写入目的向量低 32 位并清零高 96 位。项目完成了 Phase 1 环境部署与标准 RVV 基线验证、Phase 2 RTL 指令实现与波形验证，以及 Phase 3 NEMU 参考模型、RTL-NEMU 协同仿真、数学等价 RVV baseline 和受控性能评估。六类合法语义输入、非 e8 非法指令行为、标准 RVV 回归和等价 baseline 均通过验证。正式性能测试显示，compute-only 场景的稳态计算吞吐/周期加速比为 4.769517 倍，包含配置、加载、计算和存储的 end-to-end 周期加速比为 1.432995 倍。

关键词：Kunminghu V2；向量扩展；自定义指令；vdot.vv；DiffTest

一、作品概述

本项目基于 XiangShan Kunminghu V2 处理器，围绕自定义向量点积指令开展设计与验证，工作内容涵盖指令语义定义、RTL 执行逻辑实现、仿真参考模型构建及性能评估。

本项目的交付成果并非孤立的 RTL 代码，而是形成了一条贯穿自定义指令开发流程的实现与验证链路：首先建立可运行的仿真环境和标准向量指令基线；随后将自定义操作接入处理器现有的译码与向量执行资源；再通过软件参考模型复现相同的架构语义；最后使用数学等价的标准 RVV 程序进行对照测试与性能评估。该流程从功能正确性、模型一致性和性能收益三个方面，为评估自定义指令的实现效果提供了依据。

二、系统组成及设计目标

2.1 项目目标

项目目标是在 XiangShan Kunminghu V2 处理器中实现一条可验证、可复现的向量点积指令，并建立覆盖指令编码、译码、执行单元、仿真参考模型和性能

评估的完整技术闭环。在此过程中，保持现有 RVV 指令行为不变，采用标准 RVV 算法构造数学等价的对照基线，并通过 RTL 与 NEMU 的一致性验证，为项目结论提供依据。

本项目的阶段安排与赛题的三阶段交付要求逐项对应：阶段一完成运行环境搭建、运行日志记录及 vadd.vv 波形验证；阶段二完成 vdot.vv 指令语义定义、RTL 接入，以及正负数和边界条件测试；阶段三完成 NEMU 指令扩展、RTL 与 NEMU 协同验证、标准 RVV 等价实现及性能分析。各阶段交接均以前一阶段冻结的实现版本和验证证据为依据，以确保后续验证对象与 RTL 版本保持一致。

2.2 系统组成

本项目由 XiangShan RTL、NEMU 参考模型、nexus-am workload、标准 RVV 回归程序及性能 benchmark 等部分组成，并按照赛题要求分阶段推进。Phase 1 搭建可运行的处理器仿真与 DiffTest 环境；Phase 2 在后端译码逻辑和 VIMac 执行通路中接入 vdot.vv；Phase 3 将相同的指令语义同步实现到 NEMU，并使用相同测试向量验证自定义指令与标准 RVV baseline。

各组成部分承担不同职责：XiangShan RTL 是被测处理器实现，负责指令译码、功能单元调度、数据通路处理和结果提交；NEMU 作为独立的软件参考模型，用于核对处理器的架构可见状态；nexus-am workload 用于构造可在目标仿真环境中运行的测试程序；标准 RVV baseline 提供不依赖自定义操作码的功能与性能参照。波形、运行日志、`objdump` 输出、样本 CSV 和校验文件则共同构成从实现过程到验证结论的证据链。

表 1 阶段内容以及成果

阶段	主要内容	最终状态
Phase 1	搭建 XiangShan/NEMU 仿真与 DiffTest 环境；运行 Hello 程序；验证标准 RVV vadd.vv 波形	PASS
Phase 2	完成 vdot.vv RTL 接入；开展边界及正负数测试；	PASS

阶段	主要内容	最终状态
	检查执行波形	
Phase 3.1	扩展 NEMU 参考模型；完成 RTL-NEMU DiffTest 一致性验证	PASS
Phase 3.2	构建数学等价的标准 RVV baseline，并完成对照验证	PASS
Phase 3.3	基于受控 rdcycle 采样开展性能评估	PASS

2.3 技术路线

技术路线按照“建立可运行基线、实现自定义指令、开展协同验证与性能对比”的顺序推进。各阶段均保留源码、运行日志、反汇编结果、波形截图及 SHA-256 校验信息。正式报告以最终封存的证据材料为依据，不以临时重跑结果替代已确认的数据。

项目为各阶段设置了明确的验证关口：首先确认处理器、NEMU 与运行时镜像能够协同启动；随后验证现有 vadd.vv 指令行为，建立回归参照；在此基础上新增 VDOT 译码与运算逻辑，并通过 RTL 功能测试检查算术边界；之后补齐 NEMU 中的指令语义并启用 DiffTest；最后冻结两条等价 workload 和统一的性能测量协议。性能测试用于评估周期表现，不作为功能正确性的证明；功能正确性则通过相应的功能测试与一致性验证确认，而不由周期数据推断。

三、Phase 1 环境与标准 RVV 基线

3.1 仿真环境验证

Phase 1 首先在 XiangShan RTL simulator 中运行 Hello workload，并启用 NEMU reference model 和 DiffTest。运行日志显示参考模型正常加载，程序输出 Hello 信息，最终在目标 PC 命中 HIT GOOD TRAP，说明处理器仿真、内存映射、参考模型连接和基本提交路径均可用。

这一阶段的价值在于把后续开发放到真实的香山仿真环境中，而不是只在独立算术脚本中检查结果。Hello workload 能验证基本启动和提交链路；NEMU reference model 正常加载、DiffTest 开启并最终命中 GOOD TRAP，则说明后续测试具备比较 DUT 与参考模型的运行基础。该基线也为定位后续异常提供了已知可工作的起点。

The first instruction of core 0 has commited. Difftest enabled.

hello xiangshan, I am 林淮青

Core 0: HIT GOOD TRAP at pc = 0x8000014c

3.2 标准 RVV vadd.vv 回归

在 custom 指令实现前，项目使用标准 RVV vadd.vv 完成向量回归。输入 A=[1,2,3,4] 、 B=[10,20,30,40] ， 输 出 C=[11,22,33,44] ， 运 行 日 志 记 录 VECTOR_ADD_PASS 和 HIT GOOD TRAP。该回归在后续 Phase 2 与 Phase 3 中持续作为既有 RVV 行为的保护性检查。

vadd.vv 的输入输出关系简单且结果可手工复核，适合作为向量加载、向量算术、向量存储及测试程序判定逻辑的端到端检查。后续 custom 指令开发过程中保留这组回归，能够帮助区分“新增指令自身实现错误”和“既有向量通路受到意外影响”两类问题。该测试通过不代表 VDOT 已通过 DiffTest，二者在报告中按阶段分别说明。

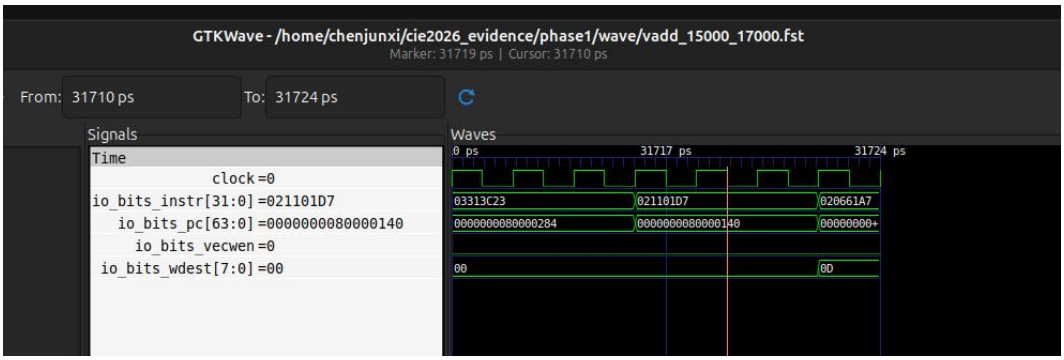


图 1 标准 RVV vadd.vv 波形证据

vsetvli t0,a3,e32,m1,ta,ma

vle32.v v1,(a0)

vle32.v v2,(a1)

vadd.vv v3,v1,v2

vse32.v v3,(a2)

四、vdot.vv 指令设计

4.1 指令语义

vdot.vv 的冻结语义为：在 VLEN=128、SEW=8 条件下，读取 vs1 和 vs2 的低 8 个字节，将每个字节解释为 signed INT8，执行 8 项有符号乘法并累加为 signed INT32。结果的 32 位位模式写入 vd[31:0]，vd[127:32] 全部清零。源向量字节 8 至 15 不参与计算，计算项数量固定为 8，不随 vl 改变。

每个 signed INT8 乘积的取值范围为 -16256 至 16384。对 8 项求和后，最大正值为 8×(-128×-128)=131072，最大负值为 8×(-128×127)=-130048。最大正值已经超过 18 位有符号整数的正向上限 131071，因此中间求和必须保留足够的符号位；本实现通过加宽加法结果并将最终和映射到 32 位结果域，避免在逐级累加过程中丢失有效位。

指令按固定 8 项计算，而不是根据 vl 动态改变乘加项数；这是当前冻结语义的一部分。它把一次操作的输入范围、运算次数和结果宽度明确限定下来，便于 RTL、NEMU、测试数据及性能 baseline 使用同一契约。对高 8 个源字节设置 poison 数据后仍应得到只由低 8 项决定的结果，这使“忽略高半区”能够被直接验证，而不是仅靠代码阅读推断。

sum = Σ(i=0..7) signed_int8(vs1.byte[i]) × signed_int8(vs2.byte[i])

vd[31:0] = uint32(sum)

vd[127:32] = 0

4.2 编码与译码

指令使用 custom-0 opcode 0x0b，bit[25] 固定为 1，属于无掩码形式。已验证的实例为 vdot.vv v10,v8,v9，对应机器码 0x0294050b，指令地址为 0x00000000800001ac。译码阶段要求 SEW=e8；当 SEW 不等于 8 时，冻结 RTL 产生 illegal instruction。

编码字段由 funct7、vs2、vs1、funct3、vd 和 opcode 共同组成；当前形式将 bit[25] 固定为 1，因此该 v0.1 指令形式不带 mask。将合法条件明确限制为

SEW=e8，是为了保证实现和参考模型均遵循赛题给定的语义边界。对其他 SEW 不执行近似计算，而是报告 illegal instruction，避免软件错误配置时产生看似有效但没有定义依据的结果。

表 2 字段以及说明

字段	值	说明
opcode	0x0b	custom-0
vd	v10	目的向量寄存器
vs1	v8	第一个源向量寄存器
vs2	v9	第二个源向量寄存器
SEW	8	唯一合法元素宽度
fuOpType	0x0e0	VIMac 内部操作类型

4.3 RTL 执行通路

RTL 修改集中在 VDotType.scala、VecDecoder.scala 和 VIMacU.scala。译码器将 custom-0 指令映射到 VDOT fuOpType；VIMacU 从两个源向量取低 8 个 signed INT8 元素，生成 8 个乘积并通过分层加法树归约。执行单元保持现有 VIMac 流水结构，结果输出到 128 位结果总线的低 32 位，高 96 位清零。RTL 证据显示该功能单元 latency 为 2 cycles，这一微架构属性与后文 benchmark 的稳态吞吐加速比分开报告。

VDOT 接入 VIMacU，而不是另建一条脱离现有向量执行框架的路径。译码器生成 VDOT fuOpType 后，执行单元从源寄存器中选取 byte[7:0]，并行形成 8 个 signed 乘积；乘积经 4、2、1 项的三级归约树合并，得到总和。平衡树把多个加法层次组织为固定结构，避免构造串行累加链；其价值在于结构清楚、项数固定且便于检查，不据此单独宣称未经测量的频率或面积收益。

普通向量算术通常通过 MGU 按元素宽度和向量控制信息形成目的向量，而 VDOT 产生的是一次归约结果，不是逐元素结果。因此实现将 VDOT 输出作为独立结果旁路选择，低 32 位承载点积位模式，高 96 位显式清零。该结果通过现有两级延迟寄存器与控制路径的 valid/控制信号对齐，避免新增功能在数据结果与提交控制之间错拍。RTL 中记录的 FU latency 为 2 cycles；这是单次执行

路径的微架构属性，不等于后续 256 次展开测试得到的吞吐周期数。

数据通路与流水对齐

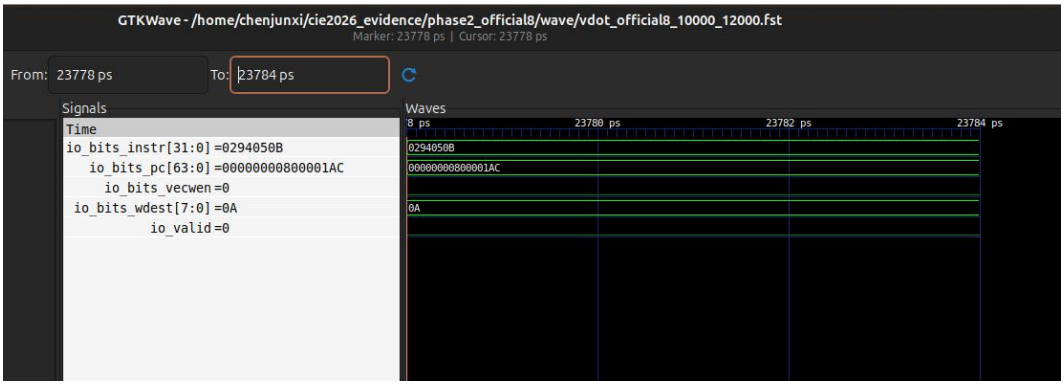


图 2　vdot.vv 指令定位波形

五、Phase 2 RTL 验证

5.1 功能测试矩阵

Phase 2 对正数、负数、最大正值、最大负值、混合符号和高半区 poison 六类输入进行了验证。测试既覆盖算术边界，也覆盖源向量高 8 字节必须被忽略的结构性约束。

六类输入分别覆盖不同的失效模式。正数和负数用小幅、易手算的值检查 signed 乘加基本路径；最大正值与最大负值检查乘积符号、累加范围和 32 位结果编码；混合符号输入检查正负乘积并存时的归约；高半区 poison 用来确认 byte[8..15] 不进入计算。非法 SEW 测试则验证译码约束，而不是合法算术结果。各类测试结合运行 PASS 标志和 GOOD TRAP，避免只依赖单一输出字符串。

表 3 测试类型以及期望结果

测试类型	期望结果	验证含义
Positive	36	基本有符号乘加
Negative	-36	负数累加与结果编码
Max positive	131072	8×(-128×-128)
Max negative	-130048	8×(-128×127)
Mixed sign	127	混合符号输入
Upper-half poison	127	高 8 字节不影响结果

5.2 波形证据

最大正值测试的波形在 23778 ps 定位到目标指令，在 23844 ps 观察到 VIMac 输入有效，fuOpType=0x0e0；23846 ps 内部结果为 0x00020000，23848 ps 输出有效并保持该结果。该值等于 8×(-128×-128)=131072，与低 8 个 signed INT8 元素的理论计算一致。

从波形时间顺序可以观察到该指令确实经过目标 VIMac 执行路径：先在 ROB 相关信号中定位 PC=0x800001ac 与机器码 0x0294050b，随后 VIMac 输入 valid 拉高并显示 fuOpType=0x0e0，经过内部结果寄存级后输出 valid 拉高。输入、操作类型、内部和输出结果在同一条波形证据链中对应起来，既说明算术结果正确，也说明目标指令进入了预期的执行功能单元。

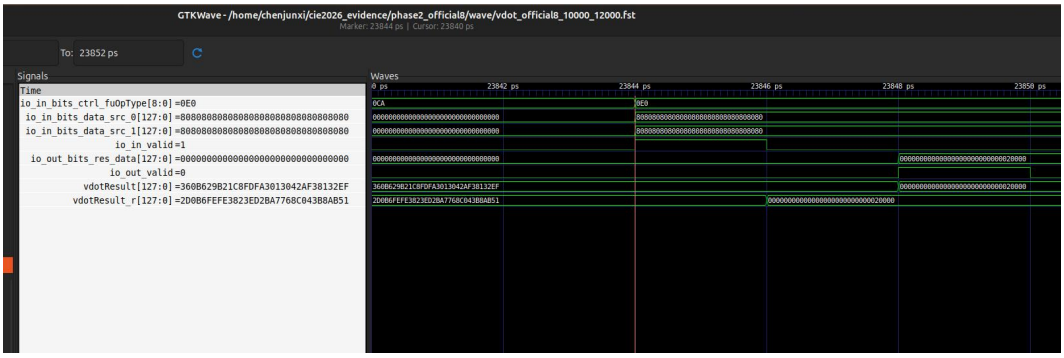


图 3　VIMac 输入与 VDOT 操作类型

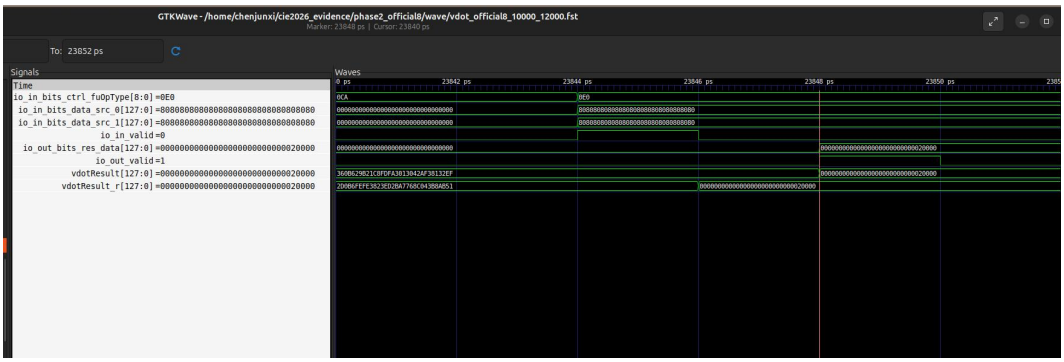


图 4　VDOT 输出结果与高位清零

5.3 标准 RVV 回归保持

Phase 2 同时保留标准 RVV vadd.vv 回归。日志中出现 Difftest enabled、VECTOR_ADD_PASS 和 HIT GOOD TRAP，说明新增 VDOT 译码与执行路径未破坏已有标准向量指令。Phase 2 的 custom VDOT 功能测试使用 --no-diff；

NEMU 参考模型协同验证在 Phase 3 单独完成。

六、Phase 3 协同仿真与参考模型

6.1 NEMU 参考模型

Phase 3 在 NEMU 中增加 custom-0 vdot.vv 的解码和执行语义，涉及 isa-all-instr.h、decode.c 和 rvv/vcompute.h。参考模型限定 SEW=e8，按已冻结 RTL 的低 8 字节、signed INT8、signed INT32 归约和高位清零规则更新向量目的寄存器，并同步更新 vector dirty 状态。最终参考模型 commit 为 55c09a3669d52ae6f6a809bab6e594939b82e512。

参考模型实现与 RTL 契约保持同源但相互独立：译码端识别 custom-0 编码，执行端以有符号 8 位读取两个源操作数的低 8 项，累计到 32 位结果，并把目的寄存器高 64 位部分清零；同时更新向量状态 dirty 标志。对于 0x0294050b，寄存器字段应解析为 vd=v10、vs1=v8、vs2=v9。点积本身具有乘法交换性，简单交换两个源寄存器仍可能得到相同算术和，因此单独核对字段索引也是必要的实现检查。

6.2 RTL-NEMU DiffTest 结果

六类合法 vdot.vv 输入均在真实 XiangShan RTL 与最终 NEMU reference model 中通过 DiffTest，并以 GOOD TRAP 结束。另行完成 SEW 非 e8 的 illegal-instruction 测试和标准 RVV vadd.vv 回归，确认 custom 指令语义、异常边界和既有 RVV 行为均满足要求。

六类合法输入覆盖从典型值到极限值的结果范围；高半区 poison 测试关注源数据选择边界；SEW 非 e8 测试关注非法指令路径；vadd.vv 回归关注既有 RVV 操作的兼容性。DiffTest 让 custom 指令的 DUT 状态能够与 NEMU 参考语义进行交叉核对，降低“RTL 与测试期望共同写错但彼此一致”的风险。测试范围仍是冻结契约规定的 vstart=0 等正式测试域，不把未测试状态外推为已支持行为。

表 4 类别以及对应的结果

类别	结果	状态

类别	结果	状态
Positive	+36	PASS
Negative	-36	PASS
Max positive	+131072	PASS
Max negative	-130048	PASS
Mixed	127	PASS
Upper-half poison	127	PASS
SEW != 8	illegal instruction	PASS
RVV vadd.vv	回归一致	PASS

6.3 验证结论

协同仿真证明了 NEMU reference model 与冻结 RTL 在正式测试域内的行为一致性。验证重点不是只比较最终算术数值，还检查了操作数索引、SEW 合法性、目的寄存器高位清零以及高半区输入忽略等架构约束。

Phase 2 的 RTL-only 测试与 Phase 3 的 RTL-NEMU 测试构成递进验证：前者先检查硬件功能是否符合预期，后者再检查硬件实现与独立软件模型是否一致。标准指令回归又提供了第三条保护线。三类证据分别回答实现正确性、模型一致性和既有功能保持问题，结论范围比单一通过标志更清晰。

七、数学等价标准 RVV baseline

7.1 baseline 指令序列

为了得到公平的性能参照，项目构造了完全不使用 custom VDOT 的标准 RVV baseline。该路径先以 e8,mf2 设置 VL=8，加载两个源向量，再使用 widening multiply 生成 INT16 乘积，最后以 widening reduction 得到 32 位有符号和。

该序列中的 widening 操作是等价性的重要部分。INT8 两数相乘可能需要超过 8 位的中间表示，vwmul.vv 将乘积扩展到 INT16；零向量作为 vwredsum.vs 的初始累加项；最后按单一 32 位结果存储。baseline 运行相同的六类输入并通过 RTL-NEMU DiffTest，说明性能对比的两侧都已在相同功能测

试域内证明可用。

```
vsetivli zero,8,e8,mf2,ta,ma
vle8.v          v8,(a0)
vle8.v          v9,(a1)
vwmul.vv        v10,v8,v9
vsetivli        zero,8,e16,m1,ta,ma
vmv.v.i         v12,0
vwredsum.vs v14,v10,v12
vsetivli        zero,1,e32,m1,ta,ma
vse32.v         v14,(a2)
```

7.2 等价性依据

Kunminghu V2 的 VLEN=128；在 SEW=8、LMUL=mf2 时，VLMAX=8，因此 baseline 恰好处理低 8 个元素。vwmul.vv 将 signed INT8 乘积 widening 为 INT16，避免 e8 中间结果溢出；vmv.v.i 提供零初值；vwredsum.vs 完成 signed widening reduction 并得到 32 位结果。baseline 反汇编中 custom VDOT 机器码计数为 0。

VLMAX 的计算为 VLEN/SEW×LMUL=128/8×1/2=8，因此 e8,mf2 正好对应 VDOT 固定使用的低 8 项，不会把高半区包含进 baseline。baseline objdump 中 custom VDOT 指令计数为 0，确保对照组确实由标准 RVV 指令组成。比较时 custom 与 baseline 计算的是同一数学函数，差别集中在核心运算指令序列及其执行成本。

7.3 baseline DiffTest

标准 RVV baseline 使用与 custom VDOT 相同的六类语义输入，全部得到一致结果，并通过 RTL-NEMU DiffTest。该结果把功能等价性和性能差异分离开来：两条路径计算相同的数学对象，区别仅在于 custom 指令将核心计算压缩为单条指令。

这组结果使性能比较具备可解释的功能前提：如果两条路径结果不同，性能数字没有比较意义；如果 baseline 本身含有 custom opcode，也无法回答标准指

令组合的成本。因而报告同时保留 DiffTest 结果和反汇编检查，分别说明“算得一样”和“确实没有调用 VDOT”。

八、性能评估

8.1 测量方法

性能测试在 XiangShan RTL 上使用 guest rdcycle 作为主指标，采用 --no-diff 运行。功能正确性已由前述 DiffTest 独立证明，因此周期测量不把 NEMU reference model 的执行计数混入 DUT 微架构周期。每条路径先执行 16 次预热，每个样本静态展开 256 次操作，共采集 5 个样本；测量区内无循环分支和 printf，custom 与 baseline 使用相同输入和 counter boundary。

测量区采用静态展开而不是循环重复，避免 loop branch 与分支控制开销进入被比较序列；不在测量区打印结果，避免串口或格式化输出扰动计数；两条路径使用相同的输入、预热次数、样本次数和 counter boundary。固定读计数器边界开销保留在原始总数中，没有对未单独测量的开销做估算扣除。采用 256 次连续操作后再按操作数归一化，降低固定边界成本对单次操作估计的相对影响。

表 5 测量范围以及包含内容

测量范围	包含内容	用途
Compute-only	源向量已加载，仅比较核心计算	衡量稳态计算吞吐
End-to-end	配置、两次 load、计算、store	衡量完整 kernel 成本

8.2 Compute-only 结果

Compute-only 测量中，custom VDOT 每次 dot product 使用 1 条指令，标准 RVV baseline 使用 5 条核心计算指令。custom 中位数为 269 cycles/256 ops，baseline 中位数为 1283 cycles/256 ops，对应每次操作 1.050781 与 5.011719 cycles。

表 6 指标以及对应结论

指标	Custom VDOT	Standard RVV	结论
Median cycles / 256 ops	269	1283	custom 更低
Cycles / op	1.050781	5.011719	4.769517x
Static instructions / op	1	5	5.000000x

8.3 End-to-end 结果

End-to-end 测量把向量配置、两次源向量加载、点积计算和结果存储全部纳入。custom 中位数为 1776 cycles/256 ops，baseline 中位数为 2545 cycles/256 ops，对应每次操作 6.937500 与 9.941406 cycles，周期加速比为 1.432995x。

表 7 指标以及结论

指标	Custom VDOT	Standard RVV	结论
Median cycles / 256 ops	1776	2545	custom 更低
Cycles / op	6.937500	9.941406	1.432995x
Static instructions / op	6	9	1.500000x

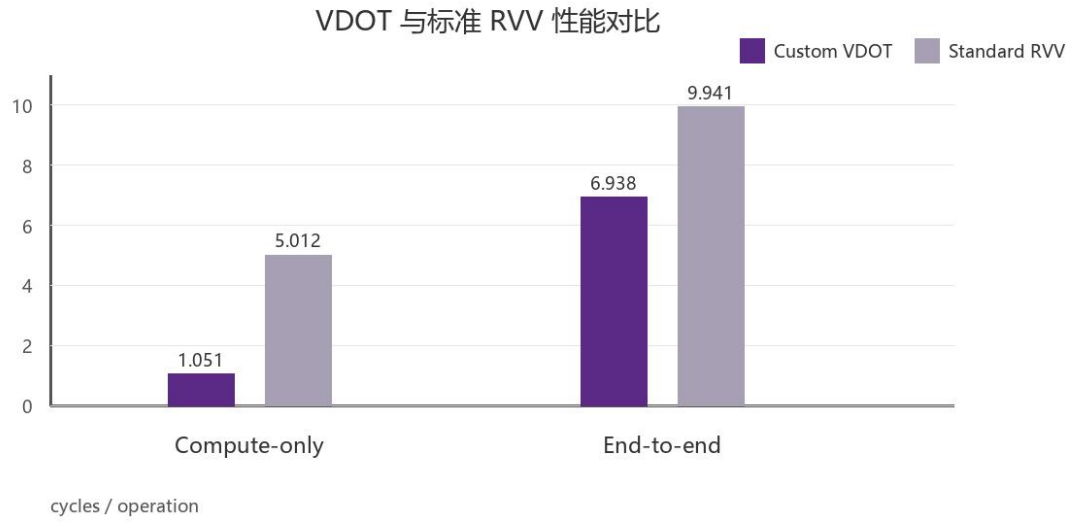


图 5 Custom VDOT 与标准 RVV cycles/op 对比

8.4 稳定性与指标解释

正式五组样本中，custom compute 为 268 或 269 cycles，baseline compute 五次均为 1283 cycles；两条路径的 end-to-end 结果分别稳定为 1776 和 2545 cycles。固定 counter boundary 开销保留在原始结果中，不进行未观测的 overhead 扣除，256 次展开操作用于摊薄固定边界开销。

五组原始样本显示，计算区 custom VDOT 在 268 至 269 cycles 之间变化，标准 RVV baseline 均为 1283 cycles；端到端两条路径在五组中分别稳定为 1776 和 2545 cycles。样本离散程度较小，说明在本次仿真设置下测得的趋势稳定。报告使用中位数计算主结果，且保留原始样本，便于复核。

表 8 Compute-only 原始周期样本

样本	VDOT 周期	RVV 周期	加速比
样本 0	269	1283	4.769517x
样本 1	269	1283	4.769517x
样本 2	268	1283	4.787313x
样本 3	269	1283	4.769517x
样本 4	268	1283	4.787313x

表 9 End-to-end 原始周期样本

样本	VDOT 周期	RVV 周期	加速比
样本 0	1776	2545	1.432995x
样本 1	1776	2545	1.432995x
样本 2	1776	2545	1.432995x
样本 3	1776	2545	1.432995x
样本 4	1776	2545	1.432995x

报告中的 4.769517x 是 256 次静态展开 dot-product 流的稳态 compute throughput/cycle speedup，不是单条 VDOT 指令的 latency speedup。VDOT 功能单元 latency 为独立的 2 cycles 设计属性；end-to-end 结果则反映配置、访存和存储共同参与后的完整 kernel 成本。

按中位数换算，compute-only 测量区的周期数从 1283 降到 269，减少约 79.0%；end-to-end 测量区从 2545 降到 1776，减少约 30.2%。核心计算静态指令数由每次 5 条降到 1 条，减少 80%；包含配置和访存的完整路径由 9 条降到 6 条，减少约 33.3%。这些比例均对应本次冻结 workload 与测量范围，不代表任意应用的端到端性能提升。

rdinstret 仅作为辅助 CSR 观察，精确静态指令数以冻结 benchmark 的 objdump 为准。whole-program cycleCnt、Guest cycle spent 和 Host time spent 含启动、打印、控制流、陷阱与模拟器开销，不能作为本项目的硬件加速比。

结果解读需要区分三种层次：2 cycles 是 RTL/VIMac 中单条功能单元的设计延迟；4.769517x 是 256 次展开核心运算流在 rdcycle 上得到的稳态计算吞吐/周期比值；1.432995x 是包含配置和访存的完整 kernel 周期比值。三者回答的问题不同，不能互相替代。报告也未对芯片面积、功耗、能效或真实应用推理时间进行测量，因此不据此声称这些指标已经改善。

九、项目总结

9.1 项目优势与适用边界

本项目的优势主要体现在实现链路完整、验证依据充分。自定义指令复用香山既有的向量执行框架，并接入 VIMac 控制路径，确保控制过程与结果延迟相匹配。同一套指令语义贯穿 RTL、NEMU 和测试程序；六类合法输入及非法 SEW 测试覆盖了算术运算、数据选择和异常边界。标准 RVV baseline 则在统一的平台、输入和计数方法下，对照验证功能等价性并评估专用指令的性能收益。由此，评审可沿着“指令编码—数据通路—功能验证—软件模型—性能测量”的链路逐项追溯项目证据。

本项目的适用范围限定如下：VDOT 已验证的语义固定为 VLEN=128、SEW=e8、低 8 个 signed INT8 元素及 vstart=0。其他元素宽度、向量长度、mask 行为和不同 vstart 情况，尚不属于本项目已验证的范围。性能数据来自 XiangShan RTL 仿真中的受控 kernel，既非 FPGA 或 ASIC 实测结果，也非完整机器学习模型的端到端推理基准。明确这些边界，有助于准确理解本次实验结果，避免将其延伸为通用性能承诺。

后续工作可在独立版本中探索参数化的项数以及对不同 SEW、VLEN 的支持，补充 mask 与 vstart 语义，研究更多累加精度组合，并开展 FPGA 或 ASIC 综合后的时序、面积和功耗评估。这些均属于后续研究方向，不属于本次已完成并封存的实现与实验结论。

项目各阶段成果与赛题要求逐项对应，关键证据已随最终提交目录保存。功能实现、协同验证和性能对照均已完成并封存；后续整理提交材料时，无需重新执行 XiangShan 或 NEMU 构建。

9.2 完成情况

项目已完成从 XiangShan/NEMU 基线环境、标准 RVV 回归、自定义 vdot.vv RTL 实现、波形定位、NEMU 参考模型、协同仿真、数学等价 baseline 到受控性能评估的完整闭环。技术证据已在最终提交目录中封存，后续工作主要是正式报告、演示材料和提交打包，不需要重新编译 XiangShan 或 NEMU。

9.3 主要结论

1. vdot.vv 在冻结语义下只计算低 8 个 signed INT8 元素，并将 signed INT32 结果写入低 32 位，高 96 位清零。

2. 六类合法输入、SEW 非法边界和标准 RVV vadd.vv 回归均通过 RTL-NEMU 或既有 DiffTest 验证。

3. 标准 RVV baseline 与 custom VDOT 在冻结测试域内数学等价，baseline 不包含 custom VDOT 编码。

4. compute-only 稳态计算吞吐/周期加速比为 4.769517x，end-to-end 周期加速比为 1.432995x。

5. VDOT FU latency 为 2 cycles，与 benchmark 的稳态 throughput 指标分属不同层次。